CPC — Lecture 6 (provisional)

Majority with insertions and deletions, Misra–Gries heavy hitters, puzzles, Binary search

Competitive Programming and Contests — Monday 05/10/2026

PROVISIONAL. The recording of L6 is not available yet. Sources: your `notes.md` **taken during the lecture of 05/10** (they fix what was covered and in which order), the professor's notes of last year (`Lessons/L5/Pearls_2025.pdf` p. 14–17 and `Binary_search_2025.pdf` p. 1–4, in this folder) and his blog note on binary search. It will be redone on the recording.
Covered on 05/10 according to your notes, in this order: majority with deletions (bit counters) → Misra–Gries → dominoes on the chessboard → the car and the fuel → binary search for the answer → social distancing (where your notes stop).
Not in your notes (kept from last year's notes / the blog, marked **[not in your notes]**): Alice & Bob, the binary search code and integer square root, exponential search. They may have been skipped or postponed.
Previous: L5 ended with Boyer–Moore, which supports insertions; deletions were announced for this lecture (`Lessons/L5/L5.pdf` §3.2).
Not included: `Binary_search_2025.pdf` p. 5–13 (two pointers, sweep line, closest pair of points, frogs and mosquitoes): in 2025 they were the following lectures.

Code: `Lessons/L6/src/lib.rs` (`cargo test -p l6`).
Exercises: `Lessons/L6/practice`, E22–E27 (`cargo test -p practice_l6`).

Contents

1 Majority element: supporting insertions and deletions **1**
1.1 Idea: one pair of counters per bit . . . 2

2 Misra–Gries heavy hitters **3**
2.1 Running time . . . 4

3 Puzzles **4**
3.1 Dominoes on a chessboard . . . 4
3.2 Alice & Bob **[not in your notes]** . . . 5

4 Binary search for the answer **5**
4.1 The car and the fuel . . . 5
4.2 The technique . . . 6
4.3 Social distancing . . . 7
4.4 Binary search: code, square root, exponential search **[not in your notes]** . . . 8

1 Majority element: supporting insertions and deletions

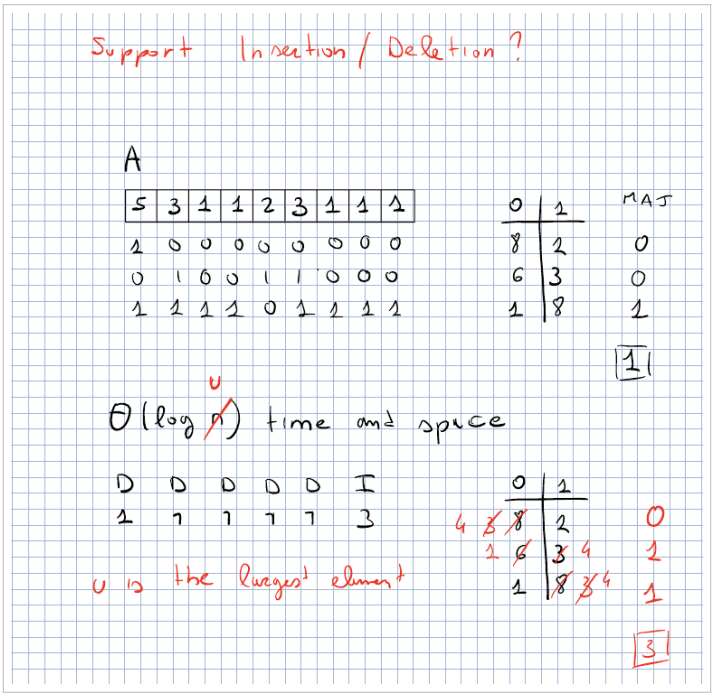
Recap from L5. Boyer–Moore keeps a candidate c and a counter: it finds the majority element (if any) in $\Theta(n)$ time and $O(1)$ space, with a second pass to verify it.

- **Insertions**: appending a new element does not break the algorithm: process it as one more step (Boyer–Moore supports dynamic insertions).

- **Deletions**: Boyer–Moore cannot undo. It does not keep any information about the elements different from the candidate, so after a deletion it does not know what the candidate should have been; it would have to be rerun on the whole array.

Problem: support insert(x), delete(x) (only of an element currently present) and "report a candidate" (correct if a majority element exists, anything otherwise), *without storing the array*.

1.1 Idea: one pair of counters per bit



Prof.'s notes of last year, `Pearls_2025.pdf` p. 14

For each bit position keep how many elements have a 0 and how many a 1 in that position. The majority element occurs more than $n/2$ times, so in *every* bit position its bit is the majority one: the i-th bit of the candidate is the most frequent value in position i.

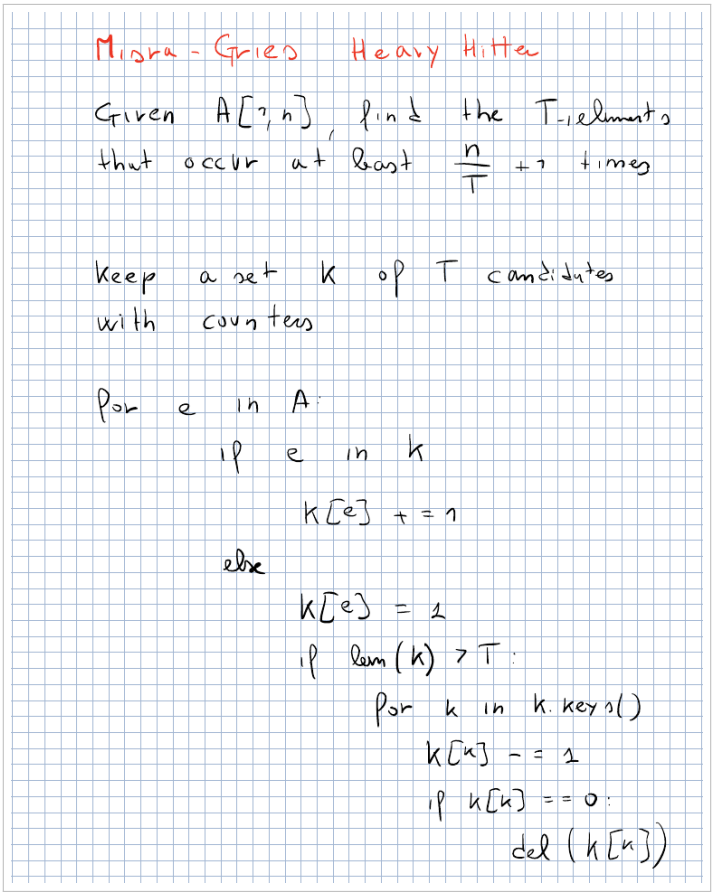
- insert(x) / delete(x): for each bit of x, increase / decrease the corresponding counter;
- report: for each position take the more frequent bit.

Example $A = [5, 3, 1, 1, 2, 3, 1, 1, 1]$ (3 bits): the counts give $001 = 1$. After deleting five 1's and inserting a 3 (the set becomes $\{5, 3, 2, 3, 3\}$) the counts give $011 = 3$.

> $O(\log u)$ time per operation and $O(\log u)$ space, where u is the largest element (number of bits). The array is never stored.

[extra] As for Boyer–Moore, if there is no majority element the candidate can be anything (it may even be a value that is not in the set).

2 Misra–Gries heavy hitters

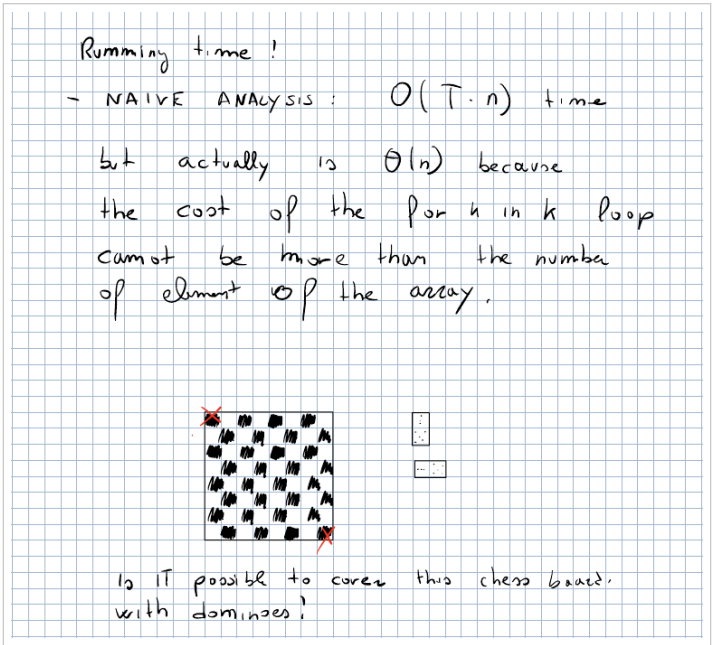


Prof.'s notes of last year, `Pearls_2025.pdf` p. 16

Generalization: given $A[1, n]$, find the elements that occur at least $\frac{n}{T} + 1$ times (there are at most $T - 1$ of them). Application mentioned in the lecture (your notes): **heavy hitters in a network**, i.e. keeping track of the senders that congest it, by source/destination IP, with memory much smaller than the stream. Keep a set K of at most T candidates with counters:

```python
for e in A:
    if e in K:
        K[e] += 1
    else:
        K[e] = 1
        if len(K) > T:
            for k in K.keys():
                K[k] -= 1
                if K[k] == 0:
                    del K[k]
```

2.1 Running time



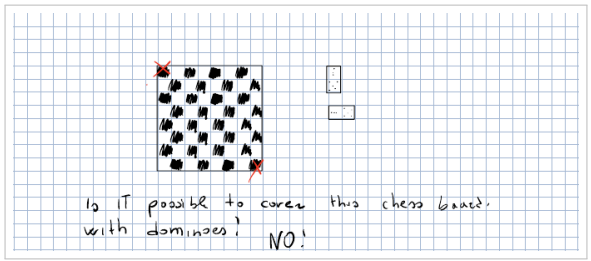
Prof.'s notes of last year, `Pearls_2025.pdf` p. 17

The naive analysis gives $O(T \cdot n)$ (the "decrease all" loop costs T and may run at every step). Actually it is $\Theta(n)$: the total cost of the "decrease all" loops cannot exceed the number of elements of the array, because each decrement cancels a previous increment and there are only n increments (amortized argument, the same kind used for the deque in L3 and for the lazy heap in L2). With $T = 2$ the threshold is $\frac{n}{2} + 1$: it is exactly the majority element problem.

$\Theta(n)$ time, $O(T)$ space. [extra] The candidates left in K may include false positives: a second pass counting them gives the exact answer (`l6::misra_gries`, `l6::heavy_hitters`).

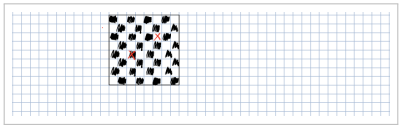
3 Puzzles

3.1 Dominoes on a chessboard



Prof.'s notes of last year, `Binary_search_2025.pdf` p. 1

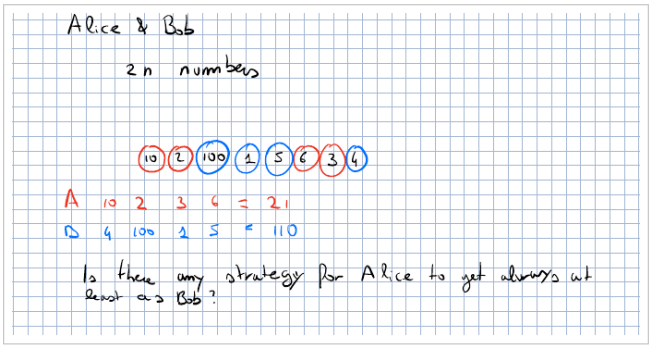
An 8×8 chessboard with two **opposite corners removed**: can it be covered with dominoes (2×1)? **No**: each domino covers one black and one white square, but the two removed corners have the same colour, so 30 squares of one colour and 32 of the other remain.



Same page: the follow-up

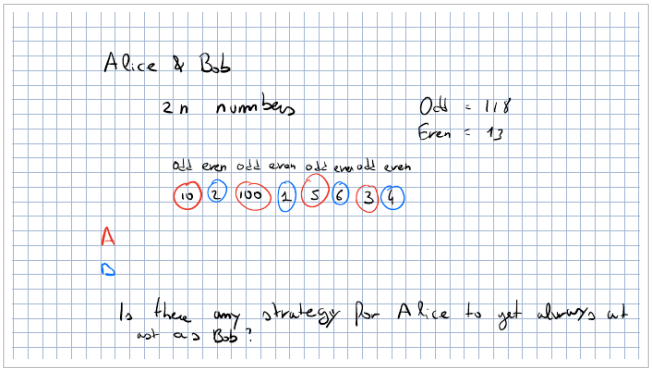
Follow-up in the notes: remove two squares of *different* colours. [extra] Then a covering always exists (Gomory's theorem: the board can be traversed by a closed path visiting every square once; removing one black and one white square cuts it into two paths of even length, each covered by dominoes along the path).

3.2 Alice & Bob [not in your notes]



Prof.'s notes of last year, `Binary_search_2025.pdf` p. 1

$2n$ numbers in a row; Alice and Bob alternately take a number from one of the two ends (Alice first); each wants the largest sum. In the notes' example $10, 2, 100, 1, 5, 6, 3, 4$ a naive play gives Alice $10 + 2 + 3 + 6 = 21$ and Bob $4 + 100 + 1 + 5 = 110$. **Is there a strategy for Alice to get always at least as much as Bob?**

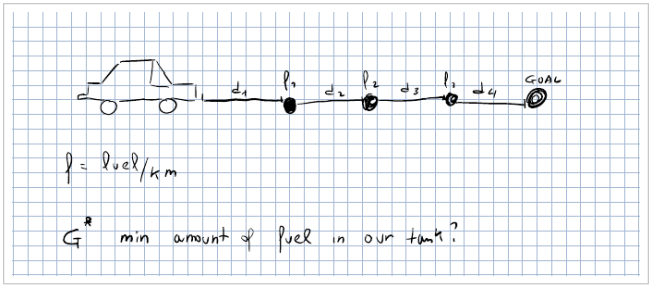


Prof.'s notes of last year, `Binary_search_2025.pdf` p. 14

Yes: sum the numbers in odd positions ($10+100+5+3 = 118$) and in even positions ($2+1+6+4 = 13$) and take all those of the larger class. Alice can always do it: if she takes position 1 (odd), both ends left are even, so Bob must take an even one, and then Alice can take an odd one again, and so on (symmetrically for even). She gets the larger of the two sums, hence at least half of the total.

4 Binary search for the answer

4.1 The car and the fuel



Prof.'s notes of last year, `Binary_search_2025.pdf` p. 4

Statement from your notes. A car starts at the beginning of a road; along the road there are points $p_1, p_2, \ldots$, the last one is the goal. We know the distances $d_1, d_2, \ldots, d_n$ (start to p_1, then between consecutive points); the car consumes F litres of fuel per km; at each point p_i it can refill f_i litres. Compute G^*, the **minimum amount of fuel in the tank at the beginning** that lets the car reach the goal. It introduced the technique of the next subsection: "with G litres at the start I reach the goal" is false for small G and true from G^* on.

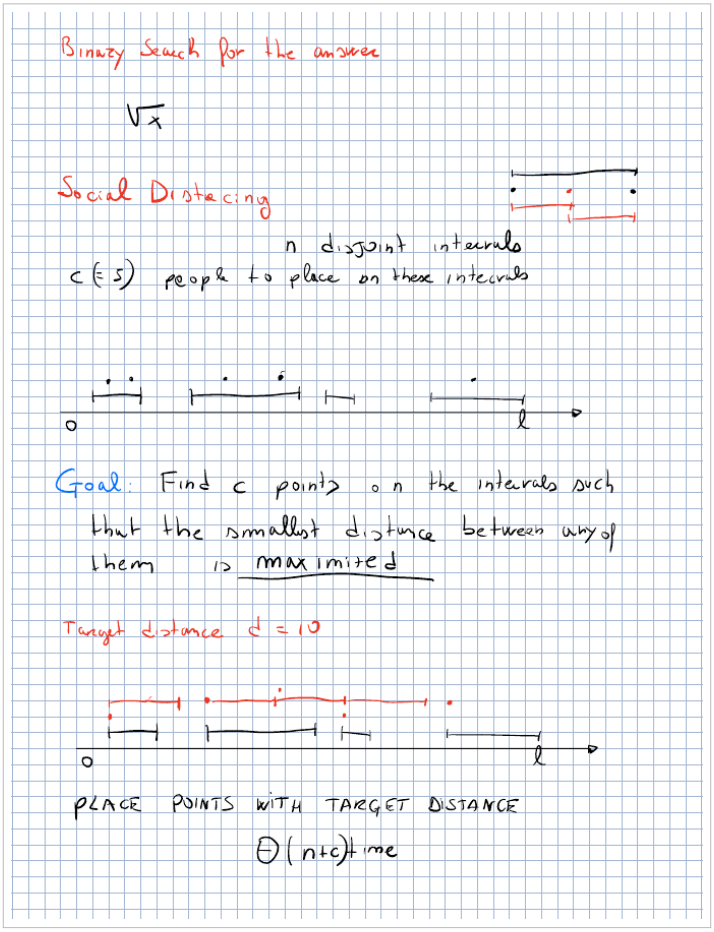
[extra] Solution written by me (how the professor solved it is not in your notes): check a given G with one scan (subtract $F\,d_i$, fail if the tank goes negative, add f_i), $\Theta(n)$, and binary search G in $[0, F\sum d_i]$: $\Theta(n \log(F\sum d_i))$. Here also a direct $\Theta(n)$ formula exists: $G^* = \max_i \big(F(d_1+\cdots+d_i) - (f_1+\cdots+f_{i-1})\big)$ (clamped at 0). Both in `l6::min_initial_fuel/min_initial_fuel_linear`, tested against brute force. Assumption: no tank capacity limit. *Unclear without the recording*: if G were instead the *capacity* of the tank (refuelling as much as fits), the same binary search on G would apply.

4.2 The technique

Your notes: at each step of the search look at the candidate answers as an array of booleans

$$[T, T, T, \ldots, T, T, F, F, \ldots, F]$$

and look for the position where T turns into F.



Prof.'s notes of last year, `Binary_search_2025.pdf` p. 2

When the candidate answers form a range and there is a **monotone predicate** (true for all values up to the answer, false after it, or vice versa), binary search on the range finds the answer with $\Theta(\log(\text{range}))$ evaluations of the predicate instead of trying all values.

```rust
fn binary_search_range<T, F>(low: T, high: T, pred: F) -> Option<T>
where
    T: Num + PartialOrd + FromPrimitive + Copy,
    F: Fn(T) -> bool,
{
    let mut low = low;
    let mut high = high;

    let mut ans = None;

    while low < high {
        let middle = low + (high - low) / FromPrimitive::from_u64(2).unwrap();

        match pred(middle) {
            true => {
                low = middle + T::one();
                ans = Some(middle)
            }
            false => high = middle,
        }
    }

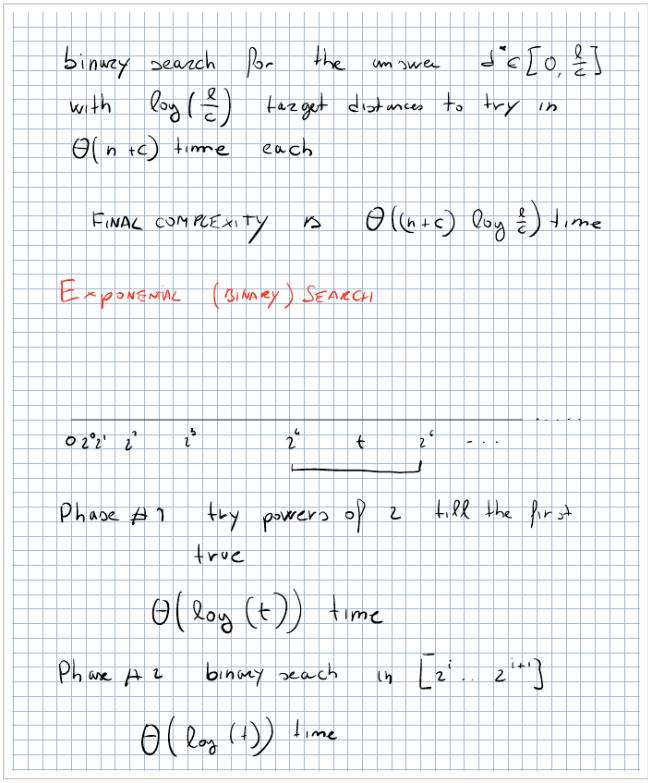
    ans
}
```

(Blog code, generic via the `num` crate; in `l6::binary_search_range` it is specialized to `u64` to avoid the dependency.)

4.3 Social distancing

n disjoint intervals of integer points on a line $[0, \ell]$ and c people ($c = 5$ in the notes): place the c people on points of the intervals so that the **smallest distance between any two of them is maximized**.

- For a **target distance** d ($d = 10$ in the notes) we can check greedily whether c people fit at distance $\geq d$: scan the sorted intervals and place each person on the leftmost valid point. $\Theta(n + c)$ time.
- The predicate "c people fit at distance $\geq d$" is monotone: if d is feasible, every smaller distance is.
- Binary search for the answer: $d > d^*$ is infeasible (F), $d \leq d^*$ feasible (T). In the lecture (your notes): start from $\ell/2$, then $\ell/4$, $\ell/8$, …: $\log \ell$ iterations, each a linear scan, so $\Theta(n \log \ell)$. Last year's notes use the slightly tighter range $[0, \ell/c]$.



Prof.'s notes of last year, `Binary_search_2025.pdf` p. 3

Social distancing: $\Theta\big((n+c)\log\frac{\ell}{c}\big)$ time (plus sorting the intervals).

[extra] **Bug in the blog code** (`select_intervals`): it searches d in $[1, l]$ where l is the *number of points* of the intervals; with gaps between intervals the answer can be larger. Example: intervals $[1,1], [4,5], [9,12], [14,16]$, $c = 2$: $l = 10$ but the answer is 15 (people at 1 and 16). The notes' bound ℓ/c uses the length of the line, which is the right one. `l6::select_intervals` is the blog code with this fix and an early stop when c people are placed (so each check is $\Theta(n+c)$); tested against brute force.

4.4 Binary search: code, square root, exponential search [not in your notes]

On a sorted array, compare the key with the middle element and keep only the half that can contain it: $\Theta(\log n)$ (divide and conquer). Code from the professor's blog note (`https://pages.di.unipi.it/rossano/blog/2023/binarysearch/`):

```rust
fn binary_search<T: Ord>(arr: &[T], key: T) -> Option<usize> {
    let mut low = 0;
    let mut high = arr.len();

    while low < high {
        let middle = low + (high - low)/2;

        match key.cmp(&arr[middle]) {
            std::cmp::Ordering::Equal   => return Some(middle),
            std::cmp::Ordering::Less    => high = middle,
            std::cmp::Ordering::Greater => low = middle + 1,
        }
    }
    None
}
```

`low + (high - low)/2` instead of `(low + high)/2`: the latter can overflow (a famous bug of real implementations). Variant: to report the **first occurrence**, on `Equal` save `ans = Some(middle)` and continue on the left with `high = middle` (`l6::binary_search_first`).

Integer square root. $\lfloor\sqrt{v}\rfloor$ is the last $x \in [0, v]$ with $x^2 \le v$, and $x \mapsto [x^2 \le v]$ is monotone: `binary_search_range(0, v + 1, |x| x * x <= v).unwrap()`. [extra] For huge v, `x * x` and `v + 1` overflow: the version in `l6::sqrt` uses `checked_mul`.

Exponential (binary) search. When the answer t has no known upper bound (or the bound is much larger than t):

- **Phase 1**: try the powers of 2, $2^0, 2^1, 2^2, \ldots$, until the predicate first becomes true (i.e. we passed t): $\Theta(\log t)$ steps;
- **Phase 2**: binary search in $[2^i, 2^{i+1}]$: $\Theta(\log t)$ steps.

Total $\Theta(\log t)$, independent of how large the universe is. [extra] `l6::exponential_search`.